AI LAB – 3

16/10/26

1BM24CS299

5K

BATCH 2

1. 8 PUZZLE – A* with Misplaced Tiles Heuristic
   CODE:

```
# 8 Puzzle using A* - Misplaced Tiles

#MISPLACED TILES
def heuristic(state, goal):
    count = 0
    for i in range(9):
        if state[i] != 0 and state[i] != goal[i]:
            count += 1
    return count


def get_moves(state):
    blank = state.index(0)
    row = blank // 3
    col = blank % 3

    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)] #UDLR
    states = []

    for move in moves:
        r = row + move[0]
        c = col + move[1]

        if 0 <= r < 3 and 0 <= c < 3:
            pos = r * 3 + c
            new_state = list(state)

            new_state[blank] = new_state[pos]
            new_state[pos] = 0

            states.append(tuple(new_state))

    return states


def astar(initial, goal):
    # Each entry: state, path, depth
    open_list = [(initial, [initial], 0)]
    best_g = {initial: 0}

    while open_list:
        # Select state with the lowest f(n)
        best_index = 0

        for i in range(1, len(open_list)):
```

```
            state, path, g = open_list[i]
            current = open_list[best_index]

            f1 = g + heuristic(state, goal)
            f2 = current[2] + heuristic(current[0], goal)

            if f1 < f2:
                best_index = i

        state, path, g = open_list.pop(best_index)

        # Ignore an outdated entry
        if g != best_g[state]:
            continue

        if state == goal:
            return path

        for new_state in get_moves(state):
            new_g = g + 1

            if new_state not in best_g or new_g < best_g[new_state]:
                best_g[new_state] = new_g
                new_path = path + [new_state]

                open_list.append((new_state, new_path, new_g))

    return None


initial = (2, 8, 3,
           1, 6, 4,
           7, 0, 5)

goal = (1, 2, 3,
        8, 0, 4,
        7, 6, 5)

solution = astar(initial, goal)

if solution:
    print("Solution found!")
    print("Number of moves:", len(solution) - 1)

    for state in solution:
        print()
        print(state[0], state[1], state[2])
        print(state[3], state[4], state[5])
        print(state[6], state[7], state[8])
```

```
else:
    print("No solution found.")
```

OUTPUT:

```
Solution found!
Number of moves: 5

2 8 3
1 6 4
7 0 5

2 8 3
1 0 4
7 6 5

2 0 3
1 8 4
7 6 5

0 2 3
1 8 4
7 6 5

1 2 3
0 8 4
7 6 5

1 2 3
8 0 4
7 6 5
```

2. 8 PUZZLE – A* with Manhattan Distance

CODE:

```
# 8 Puzzle using A* - Manhattan Distance

def heuristic(state, goal):
    distance = 0

    for i in range(9):
        tile = state[i]

        if tile != 0:
            goal_pos = goal.index(tile)

            row1 = i // 3
            col1 = i % 3

            row2 = goal_pos // 3
```

```
            col2 = goal_pos % 3

            distance += abs(row1 - row2) + abs(col1 - col2) #manhattan
distance algo


    return distance


def get_moves(state):
    blank = state.index(0)
    row = blank // 3
    col = blank % 3

    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]
    states = []

    for move in moves:
        r = row + move[0]
        c = col + move[1]

        if 0 <= r < 3 and 0 <= c < 3:
            pos = r * 3 + c
            new_state = list(state)

            new_state[blank] = new_state[pos]
            new_state[pos] = 0

            states.append(tuple(new_state))

    return states


def astar(initial, goal):
    open_list = [(initial, [initial], 0)]
    best_g = {initial: 0}

    while open_list:
        best_index = 0

        for i in range(1, len(open_list)):
            state, path, g = open_list[i]
            current = open_list[best_index]

            f1 = g + heuristic(state, goal)
            f2 = current[2] + heuristic(current[0], goal)

            if f1 < f2:
                best_index = i
```

```
        state, path, g = open_list.pop(best_index)

        if g != best_g[state]:
            continue

        if state == goal:
            return path

        for new_state in get_moves(state):
            new_g = g + 1

            if new_state not in best_g or new_g < best_g[new_state]:
                best_g[new_state] = new_g
                new_path = path + [new_state]

                open_list.append((new_state, new_path, new_g))

    return None


initial = (2, 8, 3,
           1, 6, 4,
           7, 0, 5)

goal = (1, 2, 3,
        8, 0, 4,
        7, 6, 5)

solution = astar(initial, goal)

if solution:
    print("Solution found!")
    print("Number of moves:", len(solution) - 1)

    for state in solution:
        print()
        print(state[0], state[1], state[2])
        print(state[3], state[4], state[5])
        print(state[6], state[7], state[8])
else:
    print("No solution found.")
```

OUTPUT:

```
Solution found!
Number of moves: 5

2 8 3
1 6 4
7 0 5

2 8 3
1 0 4
7 6 5

2 0 3
1 8 4
7 6 5

0 2 3
1 8 4
7 6 5

1 2 3
0 8 4
7 6 5

1 2 3
8 0 4
7 6 5
```